rsna-knee-abnormality-detection

1st Place Solution

Rank	1
Team	Team Kangaroo
Author	tascj
Topic ID	752468
Version	Original

Source: https://www.kaggle.com/competitions/rsna-knee-abnormality-detection/discussion/752468
Retrieved with Kaggle CLI on 2026-10-27.

Thanks to RSNA, the annotators and Kaggle for organizing this competition, and to my teammate @liushuzhi .

This is my part of Team Kangaroo's solution. @liushuzhi 's part is here.

0. Overview

Only 58 of the 4,407 training studies come with the 12 labels. Every study has a free-text radiology report, but the test studies do not.

So we treated the competition mainly as a **labeling competition**. We used frontier LLM agents to get different kinds of supervision for the training set: the probability of each of the 12 findings, an ROI box per finding, and per-slice lesion boxes.

1. Agentic labeling

1.1 The 12 findings

The labeling framework

Each label comes from one **non-interactive agent session**: the framework creates a separate workspace for each study (and each attempt), starts the agent in the CLI's non-interactive mode (`codex exec --json`, `claude -p --json-schema …`, `pi --print`) and collects its final answer in a given schema. Switching the CLI or the model is one line of configuration, so all comparisons below ran under the same conditions.

The workspace:

```
<workspace>/
├── AGENTS.md                  # task, definitions and grades of the 12 findings, reading
requirements, series table, full report, answer format
├── overviews/                 # an overview image per fluid-sensitive series (up to 48 slices, 6 per
row)
│   ├── SAG_FLUID.png
│   ├── COR_FLUID.png
│   └── AX_FLUID.png
├── series/
│   ├── S1_pd_tse_fs_sag/slices/001.png …
│   ├── S2_pd_tse_fs_cor/slices/001.png …
│   ├── …
│   ├── geometry.json          # DICOM geometry of each series
│   └── roles.json             # SAG_FLUID / COR_FLUID / AX_FLUID / SAG_STRUCT … → which series
├── tools/
│   ├── view_slices.py         # 1–4 slices of a series, cropped to a box and zoomed, adjustable
window
│   └── compare_location.py    # maps a point to the other series by DICOM geometry (states that the
series are not registered)
└── scratch/views/             # the only writable directory, for the output of the viewing commands
```

The answer format asks for **evidence first, then the probability**: for each finding, quotes from the report, image evidence (citing the views, series and slices looked at), a summary, and finally `present_probability`. Example output:

```
"acl": {
  "report_evidence": ["ACL is ruptured."],
```

```
  "image_evidence": [
    "VIEW 1 SAG_FLUID 14-17: markedly disorganized ACL with abnormal horizontal distal
     remnants and loss of normal bridging fibers toward the femoral attachment.",
    "VIEW 7 SAG_STRUCT 15-18: structural images confirm architectural disruption ..."],
  "present_probability": 0.98 },
"contusion": {
  "report_evidence": ["There are bone contusions in the weightbearing portion of the lateral
     femoral condyle and posterior aspect of both tibial condyle."],
  "image_evidence": [
    "VIEW 2 SAG_FLUID 8-10 ...: extensive lateral femoral and posterior lateral tibial marrow
     edema in an impaction distribution.", ...],
  "present_probability": 0.98 }
```

For this study the agent looked at 16 images with 7 tool calls; the median study took about 2.5 minutes and 15 images.

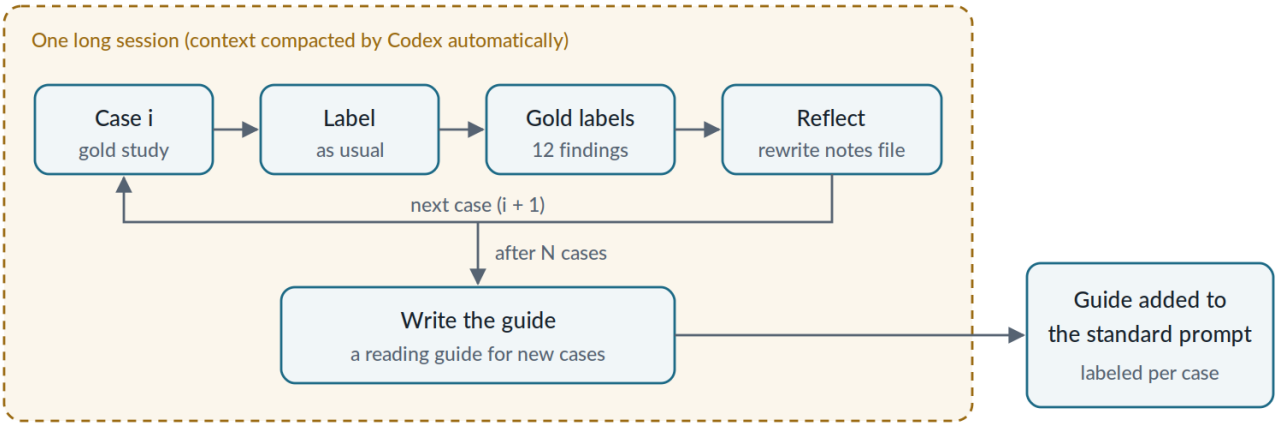
Models and protocols

I started by asking Codex to `/goal` a protocol (instructions, extensions, tools, skills, etc.) to improve AUC on the 58 gold studies, using `pi + gpt-6-luna`, then switched to `gpt-6.1-sol` to label the whole training set. After the team merge, I found that my teammate's labels were much stronger than mine. His instructions for assigning probabilities were much stronger than mine, so I switched to his protocol and to Codex for labeling.

Model	Protocol	AUC	log loss	confident errors
GPT-6 Luna	tascj's	0.904	0.555	92
GPT-6 Luna	sakaku's	0.891	0.465	61
GPT-6.1 Sol	tascj's	0.928	0.492	63
GPT-6.1 Sol	sakaku's	0.932	0.350	44
GPT-6 Astra	**sakaku's**	**0.936**	**0.329**	**35**

My lucky runs on the 58 gold studies could match the AUC of my teammate's labels, but the gaps in log loss and confident errors were large. From then on I judged labels by log loss and confident errors as well.

Trained guide



Since the protocol matters this much, I tried to 'train' a guide from the gold studies. In one long Codex session the agent labels a gold study as usual (with sakaku's protocol as the base), sees its gold labels, reflects and

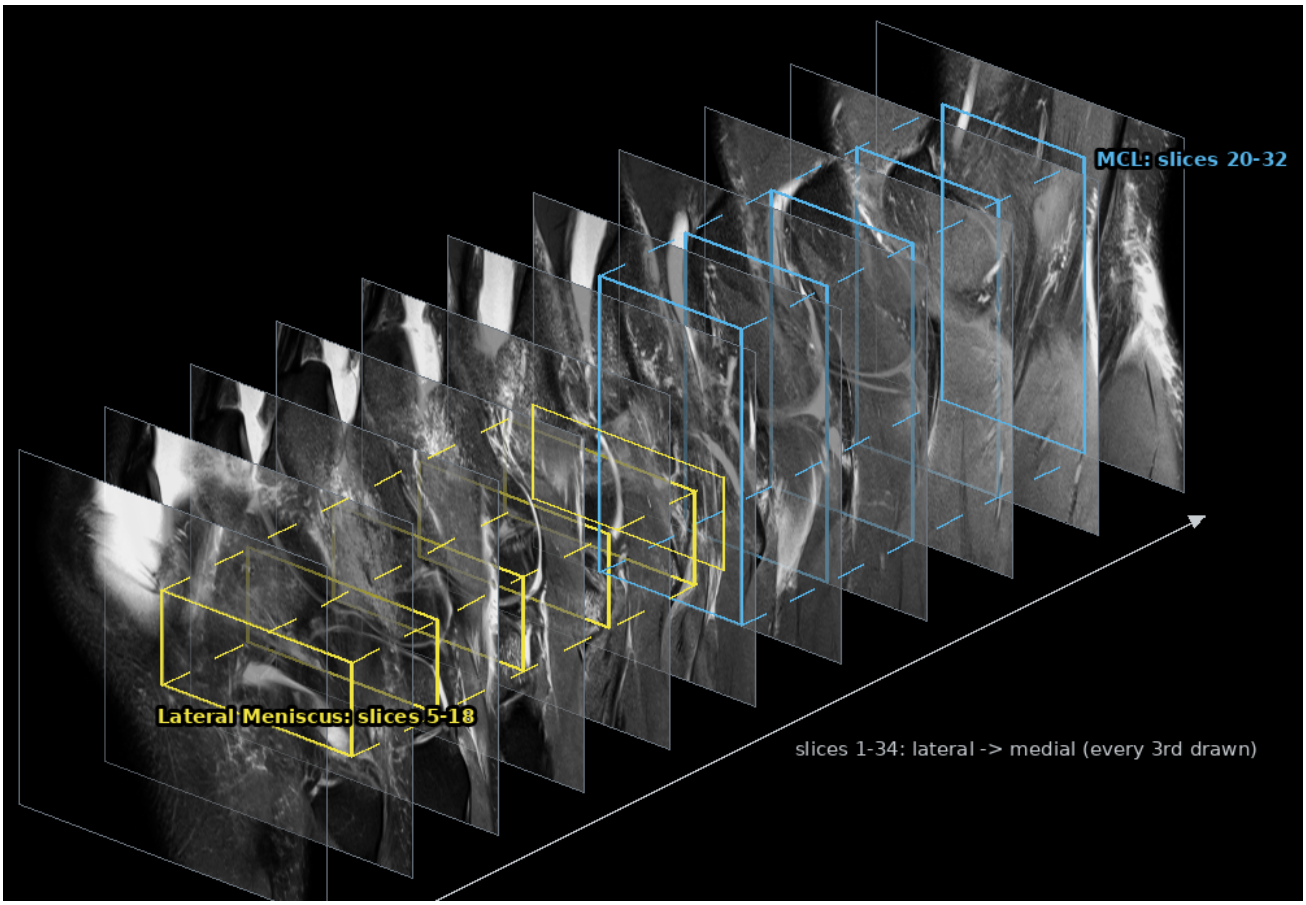
rewrites a notes file, then moves on to the next; after N cases it writes a reading guide. The guide is added to the prompt to label other studies. Scores on the 58 gold studies using `gpt-6-astra-medium`:

	AUC	log loss	confident errors
sakaku's labels	**0.936**	**0.329**	**35**
+ trained guide (5 folds, out of fold)	0.936	0.307	19
+ trained guide (learned on and applied to the same 58 studies)	0.943	0.300	16

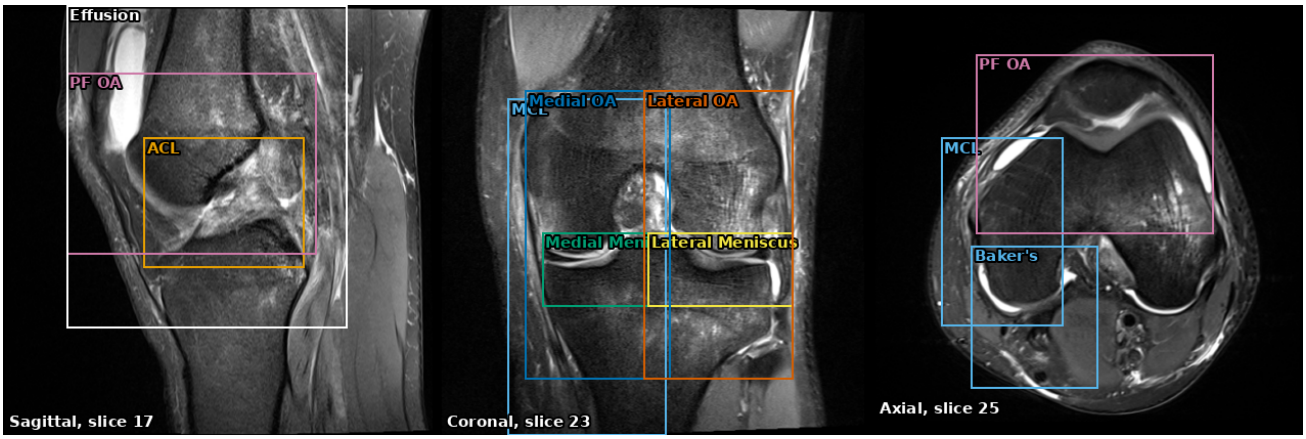
I then labeled the whole training set with the trained guide; these trained-guide labels were used to train some of my models for submisison.

1.2 ROI boxes

An ROI box is **one 3D box per series and finding** around the anatomy the finding concerns, not the lesion itself. For example, the ACL box covers the whole ligament, the PF OA box the patella and trochlea, the effusion box the recesses.



The sagittal fluid-sensitive series of the example study above (34 slices, lateral to medial, every 3rd drawn) and two of its ROI boxes: lateral meniscus (slices 5–18) and MCL (slices 20–32). Each box is one rectangle repeated over a run of consecutive slices, together a cuboid; the boxes of different findings cover different slice ranges.

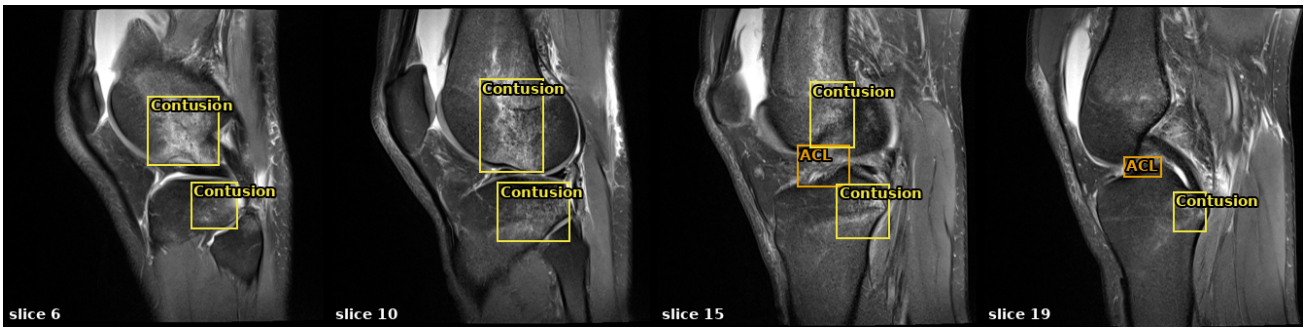


ROI boxes of the same study on three planes (each plane shows only some of the findings whose boxes cross that slice).

I used a long Codex session to get 3D ROI boxes: a main session, following the instructions, opened 21 parallel sub-sessions, each labeling a batch of studies in sequence. I got boxes for 751 studies (4,092 series). A detector was trained to predict the 12 boxes for every series at test time. The detector is a per-slice ConvNeXt-T (160 px) with 3D fusion; mean IoU 0.714 on 551 held-out series. I did not tune the detector further; its predicted ROI boxes were used in all later experiments.

1.3 Lesion boxes

Lesion boxes are **per-slice 2D boxes**: on the fluid-sensitive series, a lesion is boxed tightly on every slice where it is visible; separate parts on one slice (e.g. several fluid pockets of an effusion) get one box each.



Lesion boxes on the SAG_FLUID series of the same study: contusions (lateral femoral condyle, posterolateral tibia) and the ACL tear. The lesion boxes were drawn by GPT-6.1 Sol; Astra's answer above independently cites slices 14–17 (ACL) and 6–12 (contusion) of the same series, consistent with these positions.

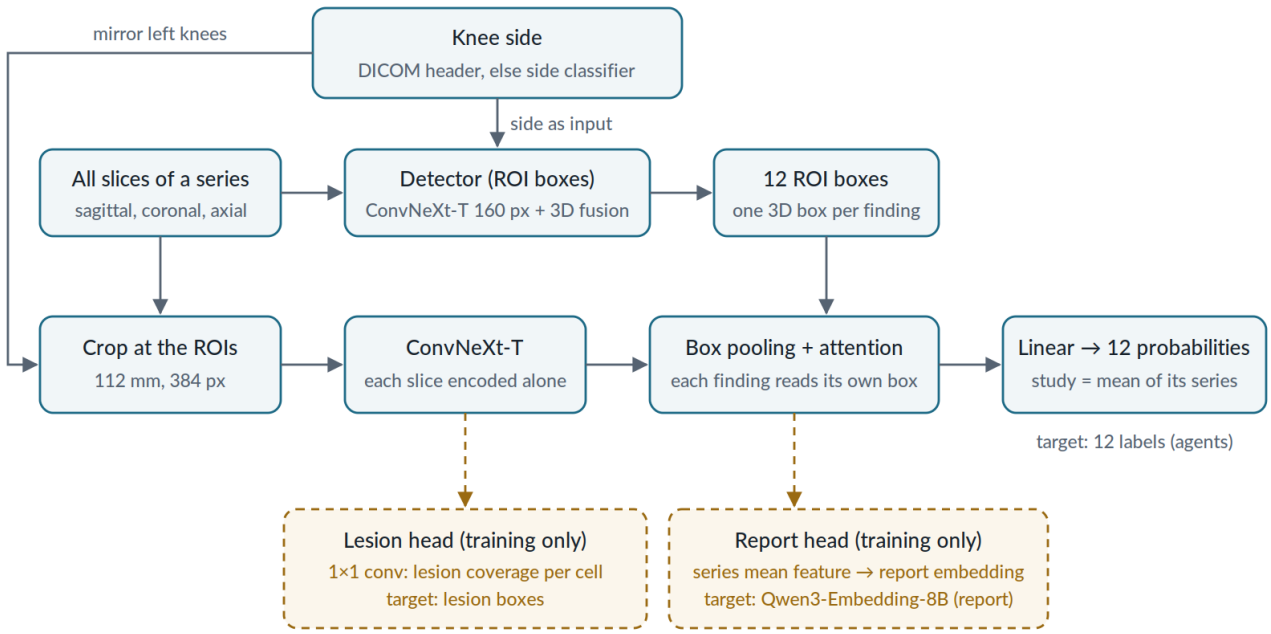
As for the 12 findings, I used one non-interactive session per study (`codex exec`, `gpt-6.1-sol-high`). Besides the two viewing commands the workspace has `check_boxes`, with which the agent checks each box before submitting: does it cover the whole lesion without too much normal tissue. I labeled 1,800 studies: 3,616 lesions, 111,029 boxes.

2. Modeling

I trained models on series; each series is assigned the labels of its study. The model takes one series as a sample with a single-slice 2D encoder; a study's prediction is the mean over its series. I did not get

improvements from study-level modeling, 2.5D or 3D. I guess the use of 3D ROI boxes makes those unimportant.

The final model structure is as follows:



Model structure. Solid: the inference path; dashed: auxiliary targets used in training only.

Input

- Each series is one sample. Every slice is resized by its pixel spacing to a 140 mm field of view (512 px) and windowed at the series' 0.5 / 99.5 percentiles; at most 64 native slices.
- Coronal and axial series of left knees are mirrored, so every knee appears as a right knee; the detector takes the side as an input instead. The DICOM headers (laterality, series description) and the agents' ROI annotations give the side for 2,853 of the 4,407 training studies; a small side classifier (ResNet18-D on the coronal and axial series, trained on those studies) gives the rest, with 99.3% study accuracy on held-out studies (RSNA 2026 Knee Side Classifier).
- A 112 mm crop centered on the enclosing box of the ACL, both menisci and the medial / lateral OA ROIs, resized to 384 px.

Using the ROIs: box pooling

- ConvNeXt-T encodes each slice into its last feature map. For finding k, each cell is weighted by its overlap with finding k's ROI box (zero outside the box's slice range); the weighted mean is finding k's series feature.
- **In-box attention:** a zero-initialized 1×1 convolution scores every cell for every finding; the weight becomes overlap $\times$ exp(score), normalized inside the box. Training starts from plain box pooling.
- Each finding reads its own feature with one linear layer; a study's probability is the mean over its series.

Lesion boxes and report text: auxiliary targets in training only

- **Lesion head** (loss weight 1): a 1×1 convolution predicts each finding's lesion coverage of every cell, with BCE on the (slice, finding) pairs that have lesion boxes.
- **Report head** (loss weight 0.3): a linear layer predicts the study's report embedding (Qwen3-Embedding-8B with a task instruction, 4096-d) from the series' mean feature; the loss is 1 − cosine. Test studies have no report, so this is the way to use it.
- Neither is used at inference, so they add no inference cost.

Training

Item	Setting
Optimizer	AdamW, one learning rate 5e-5 for all parameters, weight decay 0.02; linear warmup over 0.5 epoch, then constant; 8 epochs; EMA 0.999
Batch	8 series per update
Augmentation	per series on the GPU: scale 0.9–1.1, shift 8%, rotation 10°
Loss	BCE on the soft labels, plus the two auxiliary targets
Precision	fp16 + GradScaler, `torch.compile`

Inference

knee side (DICOM header, else side classifier) → detector ROI boxes → crop (left knees mirrored) → box pooling → mean over series

Evaluation

- **Split:** 5-fold split (by study); the 58 gold studies are kept apart, as a reference only.
- **Metric:** For each experiment, we measured AUC against multiple labels (from different models, different protocols, etc.). After collecting 20–30 public LB scores, we analyzed the correlations and decided to trust AUC(OOF, @liushuzhi 's Astra soft labels).

Gains step by step

256 px, adding one component at a time. Macro AUC of the 5-fold OOF predictions (4,349 studies) against sakaku's Astra labels (uncalibrated): soft uses the label probabilities as they are, hard thresholds them at 0.5. 58 gold: the mean of the 5 fold models on the 58 gold studies.

Step	soft	hard	58 gold
12 labels only: per-slice ConvNeXt-T	0.8721	0.9586	0.9319
+ ROI box pooling	0.8746	0.9605	0.9330
+ 112 mm crop at the ROIs	0.8756	0.9616	0.9386

Step	soft	hard	58 gold
+ in-box attention	0.8769	0.9623	0.9396
+ temperature-calibrated labels	0.8777	0.9625	0.9410
+ lesion head and report head	0.8789	0.9638	0.9432

- The temperature calibration fits $p' = \sigma(k \cdot \text{logit } p)$ on the 58 gold studies ($k \approx 0.77$), aimed at the over-confident probabilities.

3. Submissions

Efficiency track: a single model

- One ConvNeXt-T (256 px), trained-guide labels, all training data, a single checkpoint (EMA); the detector plus box pooling as above.
- **Public 0.964, private 0.954.**
- Inference: one process per T4 on two T4s; TensorRT; DICOM reading in C++; only the fluid-sensitive series are read (41% less data). With the first 1,200 training studies laid out as a test set, the whole notebook took 230 s (about 4 minutes).

Opus 5.5 one-shotted almost every optimization direction I suggested. The biggest challenge was that Kaggle cannot guarantee a stable data-reading speed: when the platform is busy, reads drop to about 30 MB/s, while when it is idle they exceed 500 MB/s. This made debugging very painful. I also don't know whether the runtime environment gets any special guarantee after clicking Submit (probably not), so I had to wait until just before the deadline, when the platform was most likely idle, to submit.

Inference notebook

Main leaderboard: an ensemble

Mainly ConvNeXt-T with input size 384, with some slightly different runs for diversity.

Run (all 384 px)	Labels	Models	Weight	Public LB	Private LB
ConvNeXt-T	trained guide	5 folds	2	0.967	0.958
MambaOut-B+	sakaku's Astra labels, calibrated (plus one head for each of the other labels from other models / protocols)	full data × 3 seeds	1		
MaxxViT-V2-B	same	full data × 3 seeds	1		
ConvNeXt-T, 6-channel	mean of sakaku's two Astra versions, calibrated	full data × 3 seeds	1		

Run (all 384 px)	Labels	Models	Weight	Public LB	Private LB
MambaOut-B+, 6-channel	same	full data × 3 seeds	1		

- The **6-channel versions** are for diversity: 6 fixed input channels, one per (plane × fat suppression), each holding the study's series of that kind, resampled in patient coordinates onto the current series' slices. Gain is more than backbone ensemble.
- All backbones use the same box-pooling recipe.

Inference notebook

NOTE

I have not fully understood every detail of the pipeline myself, especially how ROI/lession boxes are used in training. Most of the time, I ask Claude Opus 5.5, roughly, "we have these annotations available; propose several approaches to use them and run the experiments", and the gains came from that.

Check the public training notebooks for details:

- RSNA 2026 Knee Side Classifier
- RSNA 2026 Knee ROI Detector
- RSNA 2026 Knee Box-Pooling Classifier
- Labels

Supplemental Q&A

> **Takuya Mizutani · 2026-10-26T03:21:34.050000**
>
> Congratulations on the win, and thanks for sharing such a detailed write-up! If you're comfortable sharing, roughly how much did you spend on the agent-based labeling, including the finding probabilities, ROI boxes, and lesion annotations? Did you use API billing or subscription plans? A ballpark estimate would be very helpful for understanding the cost of reproducing this approach.

> **Ilia Iliev · 2026-10-26T08:40:33.623000**
>
> Congrats! Really impressive that you won both efficiency and the main track. So it is all in the labels.
>
> I tried to estimate price. Just to load the 15 images for all the series is about $220. Adding tool calls and output puts my prediction for the labeling job at $500-$1000 astra API price. Is that fair?

> **tascj · 2026-10-26T08:52:15.857000**
>
> @takuyamizutani @iliailiev1
>
> Roughly, measured in weekly quotas of the Codex 20x plan:
>
> 1. 12 labels: 3

> 2. ROI boxes: 1.5
> 3. Lesion boxes: 1.5
>
> With the debugging cost included, the actual total should be more than twice that. Codex quotas were reset quite often, so I always used up the quota as fast as possible and kept the next idea ready.